TL07 timm

Índice

1 Introducción

PyTorch Image Models (timm): colección de modelos pre-entrenados y utilidades que obtienen resultados SOTA en visión

timm en github:

- What's New: historial de cambios
- Introduction: descripción breve de timm
- Features: características / recursos del repo
  - Models: (arquitecturas de) modelos pre-entrenados con referencias
  - Optimizers: optimizadores con referencias
  - Augmentations: técnicas de aumento de datos con referencias
  - Regularization: técnicas de regularización con referencias
  - Other: otras características / recursos
- Results: tablas de resultados en ImageNet
- Getting Started (Documentation): enlace a la documentación oficial en Hugging Face
- Train, Validation, Inference Scripts: scripts estándar de entrenamiento, validación e inferencia
- Awesome PyTorch Resources recursos pytorch relacionados
- Licenses: permisivas (Apache 2.0 o similares; no GPL / LGPL)
- Citing: Ross Wightman, PyTorch Image Models

timm en HF:

- Collections: 20 colecciones de modelos; p. ej. timm Takes on the Classics
- Spaces: 6 espacios (aplicaciones web); p. ej. The timm Leaderboard
- Models: 1707+ modelos pre-entrenados
- Datasets: 15 datasets

Modelos entrenados con la librería timm: 2730+ modelos (incluidos los de la organización timm)

Documentación de la librería timm:

- *Get started:*
  - Quickstart guía de inicio
  - Installation `pip install timm` (en entorno virtual)
- *Tutorials:*
  - Feature Extraction: uso de los modelos para representar imágenes
  - Hyper-parameters (HParams): descripción de hiper-parámetros usados
  - Using the Official Training Script: en raíz de repo github; no incluidos en el paquete pip
  - Sharing and Loading Models From the Hugging Face Hub: para compartir y descargar un modelo en el HF Hub
- *Model pages:*
  - Model Summaries: resúmenes con referencias
  - Results: los resultados actualizados están en github
  - Adversarial Inception v3: primera ficha de modelo
  - ...
  - Xception última ficha de modelo
- *Referencia:* modelos, datos, optimizadores, planificadores

Objetivo: familiarizarse con timm

2 Quickstart

Lectura de un modelo pre-entrenado: create_model()

In [ ]:
```
import timm; m = timm.create_model('mobilenetv3_large_100', pretrained=True); print(str(m.eval())[:1250]+"...")
```

```
MobileNetV3(
  (conv_stem): Conv2d(3, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
  (bn1): BatchNormAct2d(
    16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True
    (drop): Identity()
    (act): Hardswish()
  )
  (blocks): Sequential(
    (0): Sequential(
      (0): DepthwiseSeparableConv(
        (conv_dw): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=16, bias=False)
        (bn1): BatchNormAct2d(
          16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True
          (drop): Identity()
          (act): ReLU(inplace=True)
        )
        (aa): Identity()
        (se): Identity()
        (conv_pw): Conv2d(16, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn2): BatchNormAct2d(
          16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True
          (drop): Identity()
          (act): Identity()
        )
        (drop_path): Identity()
      )
    )
    (1): Sequential(
      (0): InvertedResidual(
        (conv_pw): Conv2d(16, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNormAct2d(
          64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True
          (drop): Identity()
        ...
```

Listado de modelos: list_models()

- Listado de modelos con pesos pre-entrenados:

In [ ]:
```
import timm; from pprint import pprint; model_names = timm.list_models(pretrained=True); pprint(model_names[:10])
```

```
['aimv2_1b_patch14_224.apple_pt',
 'aimv2_1b_patch14_336.apple_pt',
 'aimv2_1b_patch14_448.apple_pt',
 'aimv2_3b_patch14_224.apple_pt',
 'aimv2_3b_patch14_336.apple_pt',
 'aimv2_3b_patch14_448.apple_pt',
 'aimv2_huge_patch14_224.apple_pt',
 'aimv2_huge_patch14_336.apple_pt',
 'aimv2_huge_patch14_448.apple_pt',
 'aimv2_large_patch14_224.apple_pt']
```

- Listado de modelos según patrón de nombre:

In [ ]:
```
import timm; from pprint import pprint; model_names = timm.list_models('*resne*t*'); pprint(model_names[:10])
```

```
['bat_resnext26ts',
 'cspresnet50',
 'cspresnet50d',
 'cspresnet50w',
 'cspresnext50',
 'eca_resnet33ts',
 'eca_resnext26ts',
 'ecaresnet26t',
 'ecaresnet50d',
 'ecaresnet50d_pruned']
```

Fine-tuning de un modelo pre-entrenado: create_model()

- Lectura del modelo pre-entrenado con cabeza ajustada al número de clases de la post-tarea

In [ ]:
```
model = timm.create_model('mobilenetv3_large_100', pretrained=True, num_classes=10)
```

- Cabeza a post-entrenar (como mínimo)

In [ ]:
```
model_str = str(m.eval()); print(model_str[:20]+"...\n..."+model_str[-300:])
```

```
MobileNetV3(
  (conv...
...al_pool): SelectAdaptivePool2d(pool_type=avg, flatten=Identity())
  (conv_head): Conv2d(960, 1280, kernel_size=(1, 1), stride=(1, 1))
  (norm_head): Identity()
  (act2): Hardswish()
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (classifier): Linear(in_features=1280, out_features=1000, bias=True)
)
```

In [ ]:
```
model.classifier
```

Out[ ]:
```
Linear(in_features=1280, out_features=10, bias=True)
```

Extracción de características con un modelo pre-entrenado: `model.forward_features(input)` en lugar de `model(input)`

In [ ]:
```
import timm; import torch; model = timm.create_model('mobilenetv3_large_100', pretrained=True)
x = torch.randn(1, 3, 224, 224); features = model.forward_features(x); print(features.shape)
```

```
torch.Size([1, 960, 7, 7])
```

Aumento de imágenes: timm.data.create_transform()

- Transformación genérica:

In [ ]:
```
import timm; timm.data.create_transform((3, 224, 224))
```

Out[ ]:
```
Compose(
    Resize(size=256, interpolation=bilinear, max_size=None, antialias=True)
    CenterCrop(size=(224, 224))
    MaybeToTensor()
    Normalize(mean=tensor([0.4850, 0.4560, 0.4060]), std=tensor([0.2290, 0.2240, 0.2250]))
)
```

- Transformación específica del modelo pre-entrenado:

In [ ]:
```
model.pretrained_cfg
```

Out[ ]:
```
{'url': 'https://github.com/rwightman/pytorch-image-models/releases/download/v0.1-weights/mobilenetv3_large_100_ra-f
55367f5.pth',
 'hf_hub_id': 'timm/mobilenetv3_large_100.ra_in1k',
 'architecture': 'mobilenetv3_large_100',
 'tag': 'ra_in1k',
 'custom_load': False,
 'input_size': (3, 224, 224),
 'fixed_input_size': False,
 'interpolation': 'bicubic',
 'crop_pct': 0.875,
 'crop_mode': 'center',
 'mean': (0.485, 0.456, 0.406),
 'std': (0.229, 0.224, 0.225),
 'num_classes': 1000,
 'pool_size': (7, 7),
 'first_conv': 'conv_stem',
 'classifier': 'classifier',
 'license': 'apache-2.0'}
```

- Parámetros ajustables de una transformación específica: timm.data.resolve_data_config()

In [ ]:
```
timm.data.resolve_data_config(model.pretrained_cfg)
```

Out[ ]:
```
{'input_size': (3, 224, 224),
 'interpolation': 'bicubic',
 'mean': (0.485, 0.456, 0.406),
 'std': (0.229, 0.224, 0.225),
 'crop_pct': 0.875,
 'crop_mode': 'center'}
```

- Inicialización de la transformación del modelo: timm.data.create_transform()

In [ ]:
```
data_cfg = timm.data.resolve_data_config(model.pretrained_cfg)
transform = timm.data.create_transform(**data_cfg)
transform
```

Out[ ]:
```
Compose(
    Resize(size=256, interpolation=bicubic, max_size=None, antialias=True)
    CenterCrop(size=(224, 224))
    MaybeToTensor()
    Normalize(mean=tensor([0.4850, 0.4560, 0.4060]), std=tensor([0.2290, 0.2240, 0.2250]))
)
```

- Conviene usar la transformación del modelo, tanto si coincide con la genérica como si no

Inferencia con modelos pre-entrenados:

- Lectura de una imagen para inferencia:

In [ ]:
```
import requests; from PIL import Image; from io import BytesIO
url = 'https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/timm/cat.jpg'
image = Image.open(requests.get(url, stream=True).raw); image
```

Out[ ]:

- Creación del modelo, activación del modo de evaluación y transformación del modelo

In [ ]:
```
model = timm.create_model('mobilenetv3_large_100', pretrained=True).eval()
transform = timm.data.create_transform(**timm.data.resolve_data_config(model.pretrained_cfg))
image_tensor = transform(image); image_tensor.shape
```

Out[ ]:
```
torch.Size([3, 224, 224])
```

- Inferencia: forward de la imagen

In [ ]:
```
output = model(image_tensor.unsqueeze(0)); output.shape
```

Out[ ]:
```
torch.Size([1, 1000])
```

In [ ]:
```
probabilities = torch.nn.functional.softmax(output[0], dim=0); probabilities.shape
```

Out[ ]:
```
torch.Size([1000])
```

In [ ]:
```
values, indices = torch.topk(probabilities, 5); indices
```

Out[ ]:
```
tensor([281, 282, 285, 673, 670])
```

In [ ]:
```
IMAGENET_1k_URL = 'https://storage.googleapis.com/bit_models/ilsvrc2012_wordnet_lemmas.txt'
IMAGENET_1k_LABELS = requests.get(IMAGENET_1k_URL).text.strip().split('\n')
[{'label': IMAGENET_1k_LABELS[idx], 'value': val.item()} for val, idx in zip(values, indices)]
```

Out[ ]:
```
[{'label': 'tabby, tabby_cat', 'value': 0.5101029276847839},
 {'label': 'tiger_cat', 'value': 0.22490672767162323},
 {'label': 'Egyptian_cat', 'value': 0.18352903425693512},
 {'label': 'mouse, computer_mouse', 'value': 0.006752463523298502},
 {'label': 'motor_scooter, scooter', 'value': 0.004942184779793024}]
```

3 Fine-tuning

Fine-tuning: post-entrenamiento de un modelo pre-entrenado para adaptarlo a una tarea específica

- La opción más rápida consiste en entrenar solo la cabeza (original o una nueva)
- La opción más lenta consiste en entrenar todo el modelo (con un learning rate pequeño)
- Entre ambas opciones existe la posibilidad de entrenar cabeza y parte superior del cuerpo
- `requires_grad` sirve para determinar si un parámetro es entrenable o no
- `exp` ejecuta varias épocas train-test con los lectores de datos `train_loader` y `test_loader`

In [ ]:
```python
import torch

def exp(model, device, train_loader, test_loader, lr=1e-3, epochs=15):
    loss_fn = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=lr)
    for epoch in range(epochs):
        print(f'Epoch {epoch}: train:', end=' ')
        model.train(); trsize, trbatches, trloss, tracc = 0, 0, 0, 0
        for batch in train_loader:
            X, y = batch['X'].to(device), batch['y'].to(device); trsize += len(X); trbatches += 1
            pred = model(X); loss = loss_fn(pred, y); trloss += loss.item()
            tracc += (pred.argmax(1) == y).type(torch.float).sum().item()
            loss.backward(); optimizer.step(); optimizer.zero_grad()
        trloss /= trbatches; tracc /= trsize
        print(f'loss {trloss:g} acc {tracc:.2%} test:', end=' ')
        model.eval(); tesize, tebatches, teloss, teacc = 0, 0, 0, 0
        with torch.no_grad():
            for batch in test_loader:
                X, y = batch['X'].to(device), batch['y'].to(device); tesize += len(X); tebatches += 1
                pred = model(X); teloss += loss_fn(pred, y).item()
                teacc += (pred.argmax(1) == y).type(torch.float).sum().item()
        teloss /= tebatches; teacc /= tesize
        print(f'loss {teloss:g} acc {teacc:.2%}')
```

Modelo pre-entrenado: el leaderboard de timm genera gráfica al final, p. ej. `avg_top1` en función de `param_count`

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import torch; import timm
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10).to(device)
model_str = str(model); print(model_str[:95]+"\n...\n"+model_str[-878:])
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
...
   (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (act1): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (drop_block): Identity()
      (act2): ReLU(inplace=True)
      (aa): Identity()
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (act3): ReLU(inplace=True)
    )
  )
  (global_pool): SelectAdaptivePool2d(pool_type=avg, flatten=Flatten(start_dim=1, end_dim=-1))
  (fc): Linear(in_features=2048, out_features=10, bias=True)
)
```

Post-entrenamiento: p. ej. CIFAR10

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("uoft-cs/cifar10").with_format("torch").rename_columns({'img': 'X', 'label': 'y'})
def datanorm(example):
    example['X'] = example['X'].to(torch.float) / 255.
    return example
ds = ds.map(datanorm, batched=True)
```

In [ ]:
```
train_ds = ds['train'].to_iterable_dataset(num_shards=1024)
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)
train_loader = DataLoader(train_ds, batch_size=32)
test_loader = DataLoader(test_ds, batch_size=32)
exp(model, device, train_loader, test_loader, lr=1e-3, epochs=20)
```

```
Epoch 0: train: loss 2.40844 acc 22.45% test: loss 1.86671 acc 28.32%
Epoch 1: train: loss 1.78366 acc 34.29% test: loss 1.51295 acc 43.53%
Epoch 2: train: loss 1.37014 acc 50.64% test: loss 1.31167 acc 54.83%
Epoch 3: train: loss 1.06957 acc 62.67% test: loss 1.0268 acc 64.43%
Epoch 4: train: loss 0.903757 acc 68.89% test: loss 0.948917 acc 66.99%
Epoch 5: train: loss 0.783233 acc 72.87% test: loss 1.07872 acc 62.42%
Epoch 6: train: loss 0.6803 acc 76.58% test: loss 0.847707 acc 71.15%
Epoch 7: train: loss 0.590344 acc 79.85% test: loss 0.882893 acc 70.96%
Epoch 8: train: loss 0.510733 acc 82.73% test: loss 0.877908 acc 72.31%
Epoch 9: train: loss 0.432 acc 85.27% test: loss 0.945209 acc 71.49%
Epoch 10: train: loss 0.374088 acc 87.25% test: loss 0.863961 acc 74.02%
Epoch 11: train: loss 0.325232 acc 88.91% test: loss 0.874002 acc 74.65%
Epoch 12: train: loss 0.281884 acc 90.36% test: loss 0.917744 acc 74.39%
Epoch 13: train: loss 0.244959 acc 91.80% test: loss 1.06297 acc 73.54%
Epoch 14: train: loss 0.224202 acc 92.50% test: loss 1.03745 acc 74.30%
Epoch 15: train: loss 0.214462 acc 92.77% test: loss 0.895498 acc 76.68%
Epoch 16: train: loss 0.165347 acc 94.37% test: loss 1.05016 acc 75.14%
Epoch 17: train: loss 0.156727 acc 94.76% test: loss 0.999012 acc 76.35%
Epoch 18: train: loss 0.139176 acc 95.20% test: loss 0.972918 acc 76.68%
Epoch 19: train: loss 0.144119 acc 95.18% test: loss 1.06337 acc 75.91%
```

4 Ejercicio

Ejercicio: post-entrena algún modelo timm con MNIST y FashionMNIST

MNIST:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10, in_chans=1).to(device)
```

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("ylecun/mnist").with_format("torch").rename_columns({'image': 'X', 'label': 'y'})
def datanorm(example):
    example['X'] = example['X'].to(torch.float) / 255.
    return example
ds = ds.map(datanorm, batched=True)
train_ds = ds['train'].to_iterable_dataset(num_shards=1024)
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)
train_loader = DataLoader(train_ds, batch_size=32)
test_loader = DataLoader(test_ds, batch_size=32)
exp(model, device, train_loader, test_loader, lr=1e-3, epochs=20)
```

```
Epoch 0: train: loss 0.596466 acc 88.32% test: loss 0.551226 acc 91.59%
Epoch 1: train: loss 0.340669 acc 93.60% test: loss 1.25186 acc 96.68%
Epoch 2: train: loss 0.144412 acc 96.66% test: loss 0.063143 acc 98.09%
Epoch 3: train: loss 0.0913859 acc 97.64% test: loss 0.118164 acc 96.63%
Epoch 4: train: loss 0.0835622 acc 97.86% test: loss 0.0579952 acc 98.20%
Epoch 5: train: loss 0.0947074 acc 97.85% test: loss 0.0659085 acc 98.28%
Epoch 6: train: loss 0.0807582 acc 98.12% test: loss 0.0451706 acc 98.64%
Epoch 7: train: loss 0.0689595 acc 98.40% test: loss 0.0428153 acc 98.61%
Epoch 8: train: loss 0.0422062 acc 98.79% test: loss 0.0570547 acc 98.28%
Epoch 9: train: loss 0.0417087 acc 98.81% test: loss 0.0467678 acc 98.71%
Epoch 10: train: loss 0.0393644 acc 98.90% test: loss 0.152451 acc 97.43%
Epoch 11: train: loss 0.0433075 acc 98.89% test: loss 0.0299503 acc 99.06%
Epoch 12: train: loss 0.0394538 acc 99.02% test: loss 0.0388772 acc 98.94%
Epoch 13: train: loss 0.0376909 acc 99.13% test: loss 0.0299297 acc 99.12%
Epoch 14: train: loss 0.0206368 acc 99.37% test: loss 0.0314466 acc 99.16%
Epoch 15: train: loss 0.0216556 acc 99.34% test: loss 0.0342969 acc 98.85%
Epoch 16: train: loss 0.0300563 acc 99.19% test: loss 0.0353794 acc 98.92%
Epoch 17: train: loss 0.0358975 acc 99.19% test: loss 0.0297881 acc 99.18%
Epoch 18: train: loss 0.0205992 acc 99.48% test: loss 0.0397421 acc 98.89%
Epoch 19: train: loss 0.0161028 acc 99.51% test: loss 0.0342485 acc 99.03%
```

Fashion-MNIST:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10, in_chans=1).to(device)
```

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("zalando-datasets/fashion_mnist").with_format("torch")
ds = ds.rename_columns({'image': 'X', 'label': 'y'})
def datanorm(example):
    example['X'] = example['X'].to(torch.float) / 255.
    return example
ds = ds.map(datanorm, batched=True)
train_ds = ds['train'].to_iterable_dataset(num_shards=1024)
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)
train_loader = DataLoader(train_ds, batch_size=32)
test_loader = DataLoader(test_ds, batch_size=32)
exp(model, device, train_loader, test_loader, lr=1e-3, epochs=20)
```

```
Epoch 0: train: loss 0.857072 acc 75.46% test: loss 0.669525 acc 78.85%
Epoch 1: train: loss 0.503682 acc 83.69% test: loss 0.372425 acc 86.11%
Epoch 2: train: loss 0.407668 acc 86.25% test: loss 0.346383 acc 87.11%
Epoch 3: train: loss 0.417131 acc 86.11% test: loss 0.360955 acc 86.37%
Epoch 4: train: loss 0.361645 acc 88.09% test: loss 1.0285 acc 68.71%
Epoch 5: train: loss 0.343542 acc 87.85% test: loss 0.328776 acc 87.54%
Epoch 6: train: loss 0.294496 acc 89.73% test: loss 0.296059 acc 89.16%
Epoch 7: train: loss 0.255955 acc 90.78% test: loss 0.319748 acc 88.57%
Epoch 8: train: loss 0.231364 acc 91.55% test: loss 0.289365 acc 89.92%
Epoch 9: train: loss 0.244102 acc 91.30% test: loss 0.306218 acc 89.00%
Epoch 10: train: loss 0.214887 acc 92.22% test: loss 0.297013 acc 89.47%
Epoch 11: train: loss 0.180368 acc 93.32% test: loss 0.509928 acc 89.82%
Epoch 12: train: loss 0.167133 acc 93.85% test: loss 0.328301 acc 89.06%
Epoch 13: train: loss 0.201021 acc 92.94% test: loss 0.289722 acc 90.04%
Epoch 14: train: loss 0.157227 acc 94.31% test: loss 0.271663 acc 90.68%
Epoch 15: train: loss 0.127998 acc 95.26% test: loss 0.285814 acc 91.11%
Epoch 16: train: loss 0.130141 acc 95.23% test: loss 0.29103 acc 91.32%
Epoch 17: train: loss 0.108966 acc 95.92% test: loss 0.32553 acc 89.95%
Epoch 18: train: loss 0.107626 acc 96.08% test: loss 0.321259 acc 90.57%
Epoch 19: train: loss 0.108363 acc 96.10% test: loss 0.341776 acc 90.98%
```